



## รายงานปฏิบัติการ PS07: Concurrency and Thread

### เสนอ

อาจารย์ ราชวิทย์ สโรชวิกสิต

### จัดทำโดย

|                           |             |
|---------------------------|-------------|
| นายภูมิพัฒน์ อภิวัตนะพงศ์ | 67070501035 |
| นายวิรัช ทองอุทัยศรี      | 67070501041 |
| นายเจษฎา เกียรติกมลวงค์   | 67070501080 |

รายงานนี้เป็นส่วนหนึ่งของวิชา CPE 333 Operating Systems  
ภาควิชาวิศวกรรมคอมพิวเตอร์ คณะวิศวกรรมศาสตร์  
ภาคการศึกษาที่ 1 ปีการศึกษา 2569  
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าธนบุรี

# Contents

|                                                                                    |          |
|------------------------------------------------------------------------------------|----------|
| สภาพแวดล้อมที่ใช้ในการทดลอง                                                        | 1        |
| <b>1 ภาพรวมของโปรแกรม</b>                                                          | <b>1</b> |
| 1.1 สถานการณ์ที่เลือก: บัญชีเงินฝากร่วมที่ใช้ผ่านตู้ ATM หลายตู้พร้อมกัน . . . . . | 1        |
| 1.2 โครงสร้างของโปรแกรม . . . . .                                                  | 1        |
| 1.3 การคอมไพล์ . . . . .                                                           | 3        |
| <b>2 Task 1: รันด้วย thread เดียว</b>                                              | <b>3</b> |
| 2.1 การคอมไพล์และสภาพแวดล้อม . . . . .                                             | 3        |
| 2.2 ผลการรันด้วย thread เดียว . . . . .                                            | 4        |

## สภาพแวดล้อมที่ใช้ในการทดลอง

การทดลองทั้งหมดทำบน Linux ที่ทำงานอยู่บน WSL2 ของเครื่องคอมพิวเตอร์เครื่องเดียว จุดที่มีผลต่อการทดลองนี้โดยตรงคือจำนวน CPU เพราะ thread ที่รันบน CPU คนละแกนทำงานพร้อมกันได้จริง ไม่ต้องรอให้ระบบปฏิบัติการสลับให้ รายละเอียดของสภาพแวดล้อมเป็นดังนี้

Table 1: ข้อมูลสภาพแวดล้อมของระบบที่ใช้ในการทดลอง

| รายการ          | รายละเอียด                                                                            |
|-----------------|---------------------------------------------------------------------------------------|
| ระบบปฏิบัติการ  | Ubuntu 24.04.1 LTS ทำงานบน WSL2 ของ Windows 11 Home Single Language (build 26200)     |
| Kernel          | 6.6.87.2-microsoft-standard-WSL2 (x86-64, SMP PREEMPT_DYNAMIC)                        |
| CPU             | Intel Core Ultra 9 185H ที่ Linux มองเห็นเป็น 22 logical CPU (11 core แกนละ 2 thread) |
| คอมไพเลอร์      | gcc 13.3.0 ไม่ระบุระดับ optimization จึงใช้ค่าเริ่มต้นคือ -O0                         |
| Thread library  | POSIX threads ของ glibc 2.39 (ลิงก์ด้วย -lpthread)                                    |
| เครื่องมือเสริม | taskset (util-linux) สำหรับบังคับให้ process ทำงานบน CPU ที่กำหนด                     |

**หมายเหตุเรื่องหลักฐาน:** ผลการรันทุกครั้งที่แสดงในรายงานเป็นภาพหน้าจอจริงจาก terminal ตัวเลขทุกตัวที่อ้างถึงในคำอธิบายและในตารางสรุปอ่านมาจากภาพโดยตรง ภาพทั้งหมดเก็บไว้ที่ PS07/result/screenshots/ และลำดับคำสั่งที่ใช้ถ่ายภาพอยู่ใน PS07/docs/COMMANDS.md

## 1 ภาพรวมของโปรแกรม

### 1.1 สถานการณ์ที่เลือก: บัญชีเงินฝากร่วมที่ใช้ผ่านตู้ ATM หลายตู้พร้อมกัน

ตัวอย่างในคาบเรียนใช้ thread หลายตัวบวกค่าตัวนับ (counter) ตัวเดียวกันไปในทิศทางเดียว รายงานนี้เลือกสถานการณ์ที่ต่างออกไปคือ **บัญชีเงินฝากร่วมหนึ่งบัญชี** ที่มีตู้ ATM หลายตู้ทำรายการพร้อมกัน โดยแต่ละตู้คือ thread หนึ่งตัว และยอดเงินในบัญชีคือตัวแปรที่ทุก thread ใช้ร่วมกัน

- บัญชีเปิดด้วยยอดเงิน 1,000 บาท (INITIAL\_BALANCE)
- thread ครั้งแรกเป็น **ตู้ฝากเงิน** ฝากครั้งละ 1 บาท ส่วน thread ครั้งหลังเป็น **ตู้ถอนเงิน** ถอนครั้งละ 1 บาท ทุก thread ทำรายการเท่ากันคือ tx ครั้ง
- เงินที่ไหลเข้าจึงเท่ากับเงินที่ไหลออกเสมอ **ยอดเงินสุดท้ายที่ถูกต้องต้องเท่ากับ 1,000 บาท** ไม่ว่าจะใช้กี่ thread และไม่ว่าจะทำรายการกี่ครั้ง

สถานการณ์นี้ต่างจากตัวอย่างในคาบเรียนสามข้อ ข้อแรกคือมีการเปลี่ยนค่า**สองทิศทาง** (ทั้งเพิ่มและลด) ตามที่ใบงานกำหนดให้มีทั้ง increase และ decrease ผลของ race condition จึงออกมาได้ทั้งเป็นบวกและเป็นลบ ข้อที่สองคือความผิดพลาดมีความหมายชัดเจน ถ้ายอดสุดท้ายสูงกว่า 1,000 บาทแปลว่ารายการถอนบางรายการหายไป (ธนาคารเสียเงิน) ถ้าต่ำกว่าแปลว่ารายการฝากบางรายการหายไป (ลูกค้าเสียเงิน) และข้อที่สามคือโปรแกรม**ตรวจสอบตัวเองได้** เพราะแต่ละ thread นับจำนวนรายการที่ตัวเองทำเสร็จไว้ในตัวแปร local ที่ไม่มีใครใช้ร่วม จึงพิสูจน์ได้ว่ารายการทุกรายการถูกทำจริงครบถ้วน สิ่งที่มีแต่ยอดเงินในตัวแปรที่ใช้ร่วมกันเท่านั้น

### 1.2 โครงสร้างของโปรแกรม

ตัวแปรที่ทุก thread ใช้ร่วมกันมีเพียงตัวเดียวคือ balance ซึ่งเป็นตัวแปร global จึงอยู่ในส่วน data ของ process และทุก thread ใน process เดียวกันมองเห็นตำแหน่งหน่วยความจำเดียวกัน ส่วนข้อมูลของแต่ละตู้เก็บใน struct atm หนึ่งชุดต่อหนึ่ง thread

```

1 #define INITIAL_BALANCE 1000L
2 #define AMOUNT          1L
3 #define MAX_THREADS     64
4
5 long balance = INITIAL_BALANCE;
6
7 enum role { DEPOSITOR, WITHDRAWER, BOTH };
8
9 struct atm {
10     pthread_t tid;
11     enum role role;
12     long tx;           /* transactions this thread must make */
13     long deposits;     /* written once by the thread when done: */
14     long withdrawals; /* how many transactions it really made */
15 };

```

Code 1: ตัวแปรที่ใช้ร่วมกันและข้อมูลประจำตัวของแต่ละ thread

การฝากและถอนเขียนเป็นฟังก์ชันสั้น ๆ ที่แก้ค่า balance ตรง ๆ โดยไม่มี lock หรือกลไก synchronization ใด ๆ ป้องกันไว้ ซึ่งเป็นจุดที่โง่งานกำหนดให้มี (*without proper synchronization*)

```

1 void deposit_plain(void)
2 {
3     balance = balance + AMOUNT;
4 }
5
6 void withdraw_plain(void)
7 {
8     balance = balance - AMOUNT;
9 }

```

Code 2: การฝากและถอนเงินที่ไม่มีการป้องกัน

ฟังก์ชันที่ทุก thread รันคือ atm\_thread() ซึ่งวนทำรายการตามบทบาทของตัวเอง ตัวนับ done\_dep และ done\_wdr เป็นตัวแปร local ที่อยู่บน stack ของแต่ละ thread แยกกัน จึงไม่มีทางเกิด race condition และค่าที่นับได้ถูกต้องเสมอ thread เขียนค่าเหล่านี้ก็กลับเข้า struct atm ของตัวเองเพียงครั้งเดียวตอนทำงานเสร็จ

```

1 void *atm_thread(void *arg)
2 {
3     struct atm *a = arg;
4     long done_dep = 0, done_wdr = 0;
5     long i;
6
7     if (a->role == DEPOSITOR || a->role == BOTH)
8         for (i = 0; i < a->tx; i++) {
9             deposit_plain();
10            done_dep++;
11        }
12    if (a->role == WITHDRAWER || a->role == BOTH)
13        for (i = 0; i < a->tx; i++) {
14            withdraw_plain();
15            done_wdr++;
16        }
17
18    a->deposits = done_dep;
19    a->withdrawals = done_wdr;
20    return NULL;
21 }

```

Code 3: ฟังก์ชันที่แต่ละ thread รัน

ฟังก์ชัน main() รับค่าจาก command line สองตัวคือจำนวน thread และจำนวนรายการต่อ thread (ค่าเริ่มต้นคือ 1 thread และ 1,000,000 รายการ คำสั่ง ./ps7 เหยย ๆ ตามใบงานจึงใช้ได้) จากนั้นกำหนดบทบาทของแต่ละ thread ตามตารางด้านล่าง สร้าง thread ทั้งหมดด้วย pthread\_create() ก่อนแล้วจึงรอทุกตัวด้วย pthread\_join() เพื่อให้ทุก thread ทำงานซ้อนกันได้ สุดท้ายรวมจำนวนรายการจากทุก thread มาคำนวณยอดที่ควรจะเป็น (expected) แล้วเทียบกับยอดจริง (actual)

Table 2: บทบาทของ thread ตามจำนวน thread ที่ระบุ

| จำนวน thread           | บทบาท                                                                                                |
|------------------------|------------------------------------------------------------------------------------------------------|
| 1                      | thread เดียวฝากเงิน tx ครั้งแล้วจึงถอน tx ครั้ง ใช้เป็นผลอ้างอิงแบบทำที่ละรายการตามลำดับ (Task 1)    |
| เลขคู่ตั้งแต่ 2 ถึง 64 | thread ครึ่งแรกฝากเงินอย่างเดียว ครึ่งหลังถอนเงินอย่างเดียว แต่ละตัวทำ tx รายการ (Task 2 และ Task 3) |
| เลขคี่ที่มากกว่า 1     | ไม่รับ เพราะแบ่งฝากกับถอนให้เท่ากันไม่ได้ โปรแกรมพิมพ์วิธีใช้แล้วจบการทำงาน                          |

แต่ละรอบการรันพิมพ์ผลเพียงบรรทัดเดียว เพื่อให้ภาพหน้าจอหนึ่งภาพแสดงผลของการรันซ้ำได้หลายรอบ ความหมายของแต่ละช่องเป็นดังนี้

```
[plain] threads=4 tx=1000000 in=2000000 out=2000000 | expected=1000 actual=... diff=... RACE
```

Code 4: รูปแบบผลลัพธ์หนึ่งบรรทัดต่อการรันหนึ่งรอบ

- threads และ tx คือจำนวน thread และจำนวนรายการต่อ thread ที่ระบุตอนรัน
- in และ out คือจำนวนรายการฝากและถอนทั้งหมดที่ thread นับได้จริง (จากตัวนับ local)
- expected คือยอดที่ควรเป็น คำนวณจาก  $1000 + in - out$  ซึ่งเท่ากับ 1,000 เสมอ
- actual คือค่าใน balance หลังทุก thread ทำงานเสร็จ และ diff คือ  $actual - expected$
- คำสุดท้ายเป็น ok เมื่อ diff เท่ากับศูนย์ และเป็น RACE เมื่อยอดเงินผิด

### 1.3 การคอมไพล์

คอมไพล์ด้วยคำสั่งตามใบงานทุกตัวอักษร คือ `gcc -o ps7 ps7.c -lpthread` ตัวเลือก `-lpthread` บอก linker ให้ลิงก์กับ POSIX thread library ส่วนการที่**ไม่ระบุระดับ optimization** (จึงเป็น `-O0`) เป็นความตั้งใจ เพราะถ้าใช้ `-O2` คอมไพเลอร์มีสิทธิ์มองว่าลูปที่บวก balance ทีละ 1 เป็นล่านครั้งมีผลเท่ากับการบวกครั้งเดียว และรวบทั้งลูปเหลือคำสั่งเดียว ช่วงเวลาที่ thread สองตัวจะชนกันจึงแทบหายไป ทั้งที่ได้ยังผิดอยู่เหมือนเดิม

## 2 Task 1: รันด้วย thread เดียว

ใบงานข้อ 2 ให้รันโปรแกรมด้วย thread เดียวก่อน เพื่อยืนยันว่าตรรกะของโปรแกรมถูกต้อง ถ้าผลที่ได้ในขั้นนี้ผิด ความผิดพลาดในขั้นต่อไปก็จะแยกไม่ออกว่าเกิดจาก race condition หรือเกิดจาก bug ธรรมดาในโปรแกรม

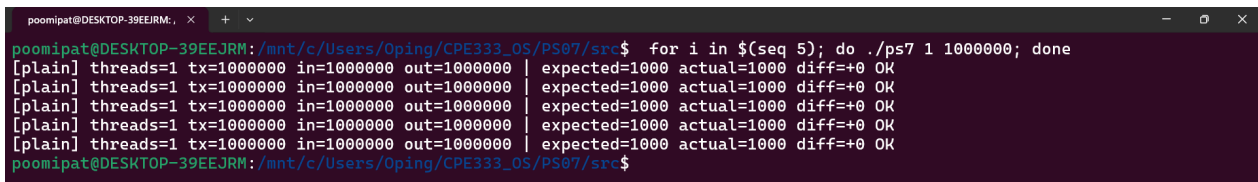
### 2.1 การคอมไพล์และสภาพแวดล้อม

```
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$ nproc; gcc --version | head -1; gcc -o ps7 ps7.c -lpthread; ls -l ps7
22
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
-rwxrwxrwx 1 poomipat poomipat 16568 Sep 27 2026 ps7
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 1: จำนวน CPU รุ่นของ gcc และการคอมไพล์ด้วยคำสั่งตามใบงาน

จากภาพ คำสั่ง `nproc` ตอบ 22 คือ Linux มองเห็น CPU 22 ตัว ซึ่งหมายความว่า thread หลายตัวของโปรแกรมนี้อาศัย CPU คนละตัวและทำงานพร้อมกันได้จริง ข้อนี้จะมีผลมากใน Task 2 คอมไพเลอร์คือ gcc 13.3.0 ของ Ubuntu คำสั่ง `gcc -o ps7 ps7.c -lpthread` ไม่พิมพ์ข้อความใดออกมาเลย แปลว่าคอมไพล์ผ่านโดยไม่มีทั้ง error และ warning และ `ls -l` แสดงไฟล์ ps7 ขนาด 16,568 byte ที่มีสิทธิ์ execute (สิทธิ์ขึ้นเป็น `rw-rw-rw-rw` ทุกช่อง เพราะไฟล์อยู่บน drive C ของ Windows ที่ WSL2 mount ไว้ที่ `/mnt/c`)

## 2.2 ผลการรันด้วย thread เดียว



```
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$ for i in $(seq 5); do ./ps7 1 1000000; done
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
[plain] threads=1 tx=1000000 in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK
poomipat@DESKTOP-39EEJRM: /mnt/c/Users/Oping/CPE333_OS/PS07/src$
```

Figure 2: Task 1: รันด้วย thread เดียว 5 รอบ ได้ยอดเงินถูกต้องทุกรอบ

คำสั่ง `./ps7 1 1000000` สร้าง thread ทำงานเพียงตัวเดียว (thread main มีหน้าที่แค่สร้างแล้วรอด้วย `pthread_join()` และไม่แตะ `balance` จนกว่า thread ทำงานจะจบ) thread ตัวนี้ฝากเงิน 1,000,000 ครั้งจนยอดขึ้นเป็น 1,001,000 บาท แล้วจึงถอนอีก 1,000,000 ครั้งจนยอดกลับลงมา

ผลทั้ง 5 รอบในภาพเหมือนกันทุกตัวอักษร คือ `in=1000000 out=1000000 | expected=1000 actual=1000 diff=+0 OK` แสดงว่า

- ตรรกะของโปรแกรมถูกต้อง ทั้งการกำหนดบทบาท การวนทำรายการ ตัวนับ local และการคำนวณยอดที่ควรเป็น
- ผลลัพธ์ **deterministic** คือรันกี่ครั้งก็ได้ค่าเดิม เพราะมี thread เพียงตัวเดียวที่อ่านและเขียน `balance` การอ่าน-บวก-เขียนแต่ละครั้งจึงจบครบก่อนเริ่มครั้งถัดไปเสมอ
- แม้ระบบปฏิบัติการจะขัดจังหวะ thread ตัวนี้กลางคันเพื่อไปรันงานอื่นได้ตลอดเวลา แต่ค่าที่ค้างอยู่ใน register ถูกเก็บไว้กับ context ของ thread และถูกคืนกลับมาครบเมื่อได้ทำงานต่อ และระหว่างนั้นไม่มี thread อื่นแก้ `balance` ค่าที่เขียนกลับจึงยังถูกต้อง

ผลนี้เป็นเส้นฐานสำหรับเทียบกับขั้นต่อไป ถ้าใช้หลาย thread แล้วยอดเงินผิดไปจาก 1,000 บาท สาเหตุจึงไม่ใช่ bug ในตรรกะของโปรแกรม แต่มาจากการที่หลาย thread แก้วตัวแปรเดียวกันพร้อมกันเท่านั้น